LAB PRACTICE · BEFORE P4

A genetic algorithm in a ring: the eight functions, in the order they run

The individual: 32 bits, two numbers

char gene[32], each element 0 or 1; double fit.
Bits 0 to 15 are x, 16 to 31 are y, most significant bit first: x and y are integers from 0 to 65535.
fitness = x*x + y*y. Maximum 8 589 672 450, all ones.
decode(gene, 0) gives x; decode(gene, 1) gives y.

One generation, two children: in this order

1 tournament twice: two random positions, the better one wins; that gives the two parents.
2 crossover_two_point, probability 0.8: swap the segment between two cuts; otherwise copy the parents.
3 mutate each child: every bit flips with probability 1/32.
4 fitness of each child: two more evaluations.
5 insert_sorted each child: it enters only if it is at least as good as the worst, in its place; the worst leaves.
The subpopulation: 64 individuals, always sorted, best at 0.

Every 20 generations: one traveller

migrate_select: the best (MIGRATE_BEST 1) or a random one.
MPI_Sendrecv: it goes to the next process, one arrives from the previous. Given; do not touch.
migrate_accept: the arrival enters with the rule of step 5.

Stop and report

Each process stops at 20 000 evaluations (10 000 generations).
Process 0 prints the best of the ring and the evaluation at which the maximum was first reached. Seed: ./P4_ga_ring 1

What each operator does to the bits

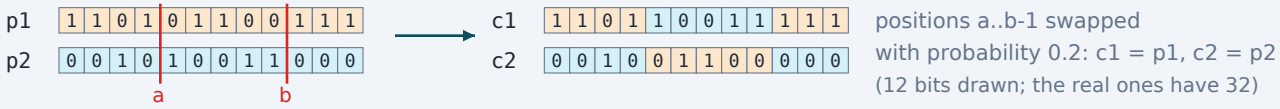
1 decode and fitness one string, two numbers



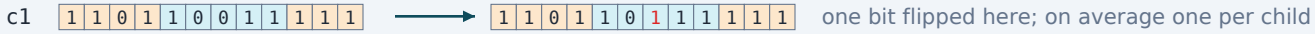
2 tournament two random positions, the better wins



3 crossover_two_point cuts a = 4, b = 9, probability 0.8



4 mutate every bit, probability 1/32



5 insert_sorted fitness of each child first, then this



6 migration round the ring every 20 generations, MPI_Sendrecv



migrate_select → to the next; from the previous → migrate_accept
the arrival enters with the rule of step 5; mpirun -np N fixes N